FILEKIT

Security Review & Technical Reference

Release assessed: **v0.1.0**

Consolidated source-level review reference

Document status	Consolidated reference draft
Repository	github.com/SmallIsBeautiful-Prog/filekit
Independent professional cryptographic audit	Not performed
Formal verification or certification	Not performed

This document consolidates recorded review findings for FileKit v0.1.0. It is not an independent audit, formal verification, or certification.

Draft for maintainer review

1. Executive summary

FileKit is a file-protection utility implemented in Rust, with native and WebAssembly/browser interfaces. It uses authenticated encryption and passphrase-based key derivation to protect file contents.

The recorded source-level review examined the cryptographic implementation and file formats, parsing and error handling, streaming operations, native output handling, command-line interface, WebAssembly/browser implementation, tests, and dependency-audit results.

Recorded conclusion: No confirmed critical vulnerability was identified in the areas examined. This is a limited conclusion about the scope of the recorded review; it is not proof that FileKit is free of vulnerabilities, and it is not an independent audit or certification.

This document consolidates the review findings available for FileKit v0.1.0. It is intended to help users and developers understand the design, recorded checks, practical usage considerations, and known limitations. It is not a substitute for examining the source code or reassessing a later release.

2. Scope and basis

The review covered the following areas, as reflected in the available consolidated record:

• Core encryption and passphrase-based key derivation.

• FKT1 and FKT2 file-format handling.

• Parsing of malformed, truncated, altered, or otherwise invalid input.

• Streaming encode/decode behavior.

• Native output publication and overwrite handling.

• Command-line and WebAssembly/browser interfaces.

• Error handling and recorded test results.

• A point-in-time dependency audit.

Evidence note: This is a consolidation of recorded conclusions, not a verbatim transcript of every review step. The available record does not preserve every individual observation or all commands and environments used. Where exact supporting detail is unavailable, this report does not invent it.

3. Cryptographic design

The recorded implementation details are:

Item	Recorded design
Authenticated encryption	AES-256-GCM
Passphrase-based key derivation	PBKDF2-HMAC-SHA-256
Default PBKDF2 iteration count	600,000
Accepted iteration-count range	100,000–2,000,000
Salt size	16 bytes, randomly generated
GCM nonce size	12 bytes

AES-GCM provides confidentiality and authentication for encrypted data when used correctly. PBKDF2 makes passphrase-based key derivation more computationally expensive than a simple fast hash. These mechanisms do not make weak passphrases strong, protect a compromised device, or by themselves prove that an implementation is free of defects.

Users should choose a strong, unique passphrase and keep it safe. FileKit cannot recover a lost passphrase.

4. File formats and compatibility

FileKit has two formats with different processing models.

Feature	FKT1	FKT2
Processing model	Whole-file	Streaming

Feature	FKT1	FKT2
Header size	36 bytes	32 bytes
Native support	Yes	Yes
Browser/WebAssembly support	Yes	Not currently recorded as supported
Authentication approach	AES-GCM authenticated encryption	Authenticated chunks and an authenticated final record

4.1 FKT1

FKT1 is used by the whole-buffer Rust API and browser/WebAssembly implementation. Native-to-browser and browser-to-native FKT1 interoperability were tested in both directions, according to the recorded project results.

Because FKT1 processes whole files in memory, practical file-size limits depend on the available memory and runtime environment.

4.2 FKT2

FKT2 is used by the Rust streaming API and native command-line encoding. The recorded design uses 4 MiB chunks, authenticates the chunks, includes an authenticated final record to detect truncation, and rejects trailing bytes. The native decoder supports FKT1 and FKT2; browser support for FKT2 was not part of the recorded implementation.

The authenticated final record and trailing-data checks are relevant to detecting incomplete or improperly extended input. They should not be interpreted as a claim that every possible malformed input or implementation defect has been ruled out.

5. Input validation and error handling

The recorded review covered rejection or handling of invalid inputs, including:

• Malformed headers and invalid format parameters.

• Iteration counts outside the accepted range.

• Incorrect passphrases and authentication failures.

• Tampered or corrupted input.

• Truncated streaming input.

• Trailing data after an encoded stream.

The test record includes wrong-passphrase and tampering/corrupted-input cases. This summary reports the coverage recorded; it does not claim exhaustive testing of every possible input or failure mode.

6. Output handling and API usage

6.1 Native output handling

The recorded native safe-output design writes to a temporary file in the destination directory, flushes and synchronises output, then uses hard-link publication that fails if the destination already exists. This is intended to avoid overwriting an existing destination and to prevent incomplete output from being published as the final file.

The precise guarantees still depend on operating-system and filesystem behavior. The recorded review did not establish identical guarantees across every platform or filesystem.

6.2 Streaming decoder: important caller responsibility

Important: `decode_reader` **may write partial plaintext to the supplied writer before the complete encrypted input has been validated and authenticated.**

Callers should therefore write decoded output to a temporary destination and publish it only after decoding completes successfully. If decoding fails, discard the incomplete output. Do not write directly to a final destination when partial output must not be exposed.

This is an API usage consideration that callers need to account for; it is not, by itself, evidence of a cryptographic break.

6.3 Browser and WebAssembly behavior

The browser implementation uses a browser-managed download flow. WebAssembly in the browser cannot freely choose arbitrary filesystem paths or provide the same native temporary-file and no-overwrite publication behavior.

The browser implementation processes whole files in memory, so large files may exceed practical memory limits. Browser behavior and download handling are also subject to the browser and operating system.

7. Recorded tests and checks

The available project record reports the following results for the v0.1.0 development/release baseline:

• **Rust tests:** 30 passed — 19 unit tests and 11 integration tests.

• **Native check:** `cargo check` passed.

• **WebAssembly target check:** `cargo check --target wasm32-unknown-unknown --lib` passed.

• **WebAssembly package build:** `wasm-pack build --target web --out-dir web/pkg` passed.

• **Browser interface:** reported as tested and working.

• **FKT1 interoperability:** native-to-browser and browser-to-native tests passed in both directions.

• **Dependency audit:** 61 dependencies were scanned; the recorded result was zero known vulnerabilities and zero warnings at the time of the scan.

These are recorded results, not a guarantee that the same checks will pass against every later commit, environment, or dependency state. The dependency-audit result is time-sensitive: new advisories may appear after a scan.

This report does not independently rerun these checks. It records the results preserved in the project review history.

8. Limitations and residual risks

The following limitations are important when deciding whether and how to use FileKit:

• **Passphrase guessing:** A weak or reused passphrase may be vulnerable to guessing. Use a strong, unique passphrase.

• **Lost passphrase:** There is no recovery mechanism for a lost passphrase.

• **Metadata:** Encryption does not conceal all metadata. File size and surrounding context may remain observable.

• **Compromised endpoint:** Malware, a compromised operating system, or access to a running process may expose plaintext or secrets.

• **Memory and zeroisation:** Secret-handling and zeroisation precautions can reduce exposure, but cannot guarantee that every copy of secret material is erased from every memory location.

• **Browser memory:** Whole-file browser processing can be impractical for large files.

• **Different output guarantees:** Browser-managed downloads do not provide the same output-publication controls as the native implementation.

• **Streaming API use:** Callers must discard partial output if `decode_reader` fails and must publish output only after successful completion when atomic final-output behavior is required.

• **Dependency changes:** A clean dependency audit is only a point-in-time result.

• **Review scope:** The recorded review was not an independent professional cryptographic audit, formal verification, or certification.

Users should consider their threat model and operating environment before relying on FileKit for high-consequence or regulated use.

9. Conclusion

The recorded source-level review of FileKit v0.1.0 examined important security-relevant parts of the implementation and recorded no confirmed critical vulnerability in the areas examined. The project also recorded successful tests, build checks, browser testing, cross-implementation FKT1 interoperability, and a dependency audit with no known vulnerabilities or warnings at the time it was run.

Those findings are useful evidence of development and review work, but they do not establish that FileKit is free of all vulnerabilities. The streaming decoder's partial-output behavior, passphrase quality, endpoint security, browser resource limits,

and the time-sensitive nature of dependency audits remain relevant considerations.

This document is a consolidated technical reference based on the recorded review conclusions. It should not be described as an independent audit or certification. Changes to FileKit after v0.1.0 may require new review and testing before these conclusions can be applied to later releases.

10. Document maintenance

• **Release assessed:** FileKit `v0.1.0`

• **Document status:** Draft for maintainer review

• **Recommended repository path:** `docs/SECURITY-REVIEW.md`

• **Recommended PDF filename:** `docs/SECURITY-REVIEW.pdf`

When FileKit changes materially, review this document and clearly identify any new review scope, test results, or changes to security-relevant behavior. Do not update the assessed release or date in a way that suggests a later version was reviewed unless that review actually occurred.